Core Java

○ Introduction to Java

1. History of Java

- Java was developed by **James Gosling** at **Sun Microsystems** in **1991**.
- The project was first called the **Green Project**.
- The original name of Java was **"Oak."**
- Later, it was renamed **"Java"** after a type of coffee.
- **Java 1.0** was officially released in **1995**.
- It was created to make programs that could run on **any device** (platform-independent).
- Java was mainly used for **internet applications** and **network programming**.
- Java's main motto: **"Write Once, Run Anywhere."**

2. Features of Java (Platform Independent, Object-Oriented, etc.)

- **Simple** – Easy to learn and understand.
- **Object-Oriented** – Everything in Java is based on objects and classes.
- **Platform Independent** – Java code runs on any device having JVM (Write Once, Run Anywhere).
- **Portable** – Java programs can be easily moved from one system to another.
- **Secure** – Java provides security through byte code verification and no direct memory access.
- **Robust** – Strong memory management and error handling.
- **Multithreaded** – Can perform multiple tasks at the same time.
- **High Performance** – Uses **Just-In-Time (JIT)** compiler for fast execution.
- **Distributed** – Supports distributed applications using technologies like RMI and EJB.
- **Dynamic** – Can load classes at runtime and supports dynamic linking.

3.Understanding JVM, JRE, and JDK

1. JDK (Java Development Kit)

- Used to **develop and run** Java programs.
- Contains **JRE + development tools** (compiler, debugger, etc.).
- Needed by **developers**.
- Includes **javac (compiler)**.

2. JRE (Java Runtime Environment)

- Used to **run** Java programs.

- Contains **JVM + library classes**.
- Does **not** include development tools.
- Needed by **users** to run Java applications.

3. JVM (Java Virtual Machine)

- Responsible for **executing Java bytecode**.
- Provides **platform independence**.
- Converts **bytecode into machine code**.
- It is **part of JRE,** not a physical machine.

4. Setting up the Java Environment and IDE (e.g., Eclipse, IntelliJ).

Step 1: Install Java

1. Download **JDK** (Java Development Kit)
2. Install it
3. Open Command Prompt and type: java -version

   (If version shows → Java installed)

Step 2: Install IDE (Eclipse / IntelliJ)

1. Download **Eclipse** or **IntelliJ IDEA**
2. Install and open it
3. Choose a **workspace folder**

Step 3: Create Java Project

1. Click **New Project**
2. Select **Java**
3. Enter project name
4. Click **Finish**

Step 4: Create Java Class

1. Right-click **src folder**
2. New → Java Class
3. Enter class name
4. Click **OK**

Step 5: Run Program

1. Write code
2. Click **Run**
3. Output appears on screen

5. JAVA PROGRAM STRUCTURE (PACKAGES, CLASSES, METHODS)

1. Package

- A **package** in Java is used to **group related classes, interfaces, and sub-packages** together.
- It helps in **organizing code** and **preventing name conflicts** between classes.
- Java provides **built-in packages** (like java.util, java.io, java.lang) and you can also create **user-defined packages**.
- Using packages makes large projects easy to manage and maintain.
- Syntax:

```
package  packagename;
```

2. Class

- A **class** is the **basic building block** of a Java program.
- It is a **blueprint** for creating objects.
- A class contains **variables (data members)** and **methods (functions)** that define the properties and behavior of an object.
- Every Java program must have **at least one class**.
- Objects are created using the `new` keyword.
- Syntax:

```
class ClassName
{
   // variables/properties
   // methods
}
```

3. Method

- A **method** is a **block of code** that performs a specific task.
- Methods help in **reusing code** and **improving readability**.
- Each method has a **return type, name,** and **parameters** (optional).
- Syntax:

```
returnType  methodName(parameters)
{
   // body of the method
}
```

4. Main Method

- The **main() method** is the **starting point** of every Java program.
- The JVM calls the main method first when the program starts execution.
- It must always be written as:

```
public static void main(String[] args)
```

- **public** → accessible by JVM
- **static** → no need to create an object
- **void** → does not return any value
- **String[] args** → stores command-line arguments

```
package mypackage;    // package declaration
public class HelloWorld {  // class declaration
   void displayMessage() {  // method declaration
      System.out.println("Welcome to Java Programming!");
   }
   public static void main(String[] args) {  // main method
      HelloWorld obj = new HelloWorld();  // object creation
      obj.displayMessage();  // method call
   }
}
```

6. Primitive Data Types in Java (int, float, char, etc.)

- In Java, **primitive data types** are the **basic building blocks** of data manipulation.
- They store **simple values** (not objects).
- There are **8 primitive data types,** grouped into 4 categories:

1. Integer Types

Data Type	Size	Default Value	Range	Example
byte	1 byte (8 bits)	0	-128 to 127	`byte a = 10;`
short	2 bytes (16 bits)	0	-32,768 to 32,767	`short s = 1000;`
int	4 bytes (32 bits)	0	-2,147,483,648 to 2,147,483,647	`int n = 50000;`
long	8 bytes (64 bits)	0L	Very large range	`long l = 100000L;`

2.Floating-Point Types

Data Type	Size	Default Value	Range	Example
float	4 bytes	0.0f	~3.4e−038 to 3.4e+038	`float f = 3.14f;`
double	8 bytes	0.0d	~1.7e−308 to 1.7e+308	`double d = 99.99;`

3. Character Type

Data Type	Size	Default Value	Range	Example
char	2 bytes	`'\u0000'`	0 to 65,535 (Unicode)	`char c = 'A';`

4. Boolean Type

Data Type	Size	Default Value	Possible Values	Example
boolean	1 bit (JVM dependent)	false	`true` or `false`	`boolean isJavaFun = true;`

7. Variable Declaration and Initialization

- A **variable** is a named memory location used to store data that can change during program execution.
- Declaring a variable means **telling the compiler** what kind of data the variable will hold.
- Syntax:

```
dataType variableName;
```

- Example:

```
int age;
float marks;
```

- Initializing a variable means **assigning an initial value** to it.

Syntax:

```
variableName = value;
```

8. Operators: Arithmetic, Relational, Logical, Assignment, Unary, and Bitwise

- Operators are **symbols** used to perform operations on variables and values.

1. Arithmetic Operators

- Used for **mathematical calculations**.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (remainder)	10 % 3	1

2. Relational (Comparison) Operators

- Used to **compare two values**.
- Result → true or false.

Operator	Meaning	Example	Result
==	Equal to	a == b	false
!=	Not equal to	a != b	true
>	Greater than	a > b	true
<	Less than	a < b	false
>=	Greater or equal	a >= b	true
<=	Less or equal	a <= b	false

3. Logical Operators

- Used to **combine multiple conditions**.

Operator	Meaning	Example	Result
&&	Logical AND	(a > b && b < 30)	true if **both** true
`	Logical OR	`	Logical OR
!	Logical NOT	!(a > b)	reverses result

4. Assignment Operators

- Used to **assign values** to variables.

Operator	Example	Meaning
=	a = 10	assign 10 to a
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

5.Unary Operators

- Used with **a single operand**.

Operator	Meaning	Example	Result
++	Increment by 1	++a	increases value
--	Decrement by 1	--a	decreases value
+	Positive sign	+a	same value
-	Negative sign	-a	negates value
!	Logical NOT	!true	false

6. Bitwise Operators

Used to perform operations on **bits** (0s and 1s).

Operator	Meaning	Example	Result
&	AND	5 & 3	1
`	`	OR	`5
^	XOR	5 ^ 3	6
~	NOT	~5	-6
<<	Left shift	5 << 1	10
>>	Right shift	5 >> 1	2

9. TYPE CONVERSION AND TYPE CASTING

- Type Conversion means **automatically or manually changing** one data type into another.

- In Java, there are **two types** of conversions:

  1.Implicit Conversion (Type Conversion / Widening)

  2.Explicit Conversion (Type Casting / Narrowing)

1 .Implicit Type Conversion (Widening Conversion):

- When **Java automatically converts** a smaller data type into a larger one — **no data loss** occurs.
- Example:

```
int num = 10;
double result = num;   // int → double (automatically)
System.out.println(result);  // Output: 10.0
```

2. Explicit Type Casting (Narrowing Conversion)

- When **you manually convert** a larger data type into a smaller one using **casting syntax**.
- Syntax:

```
dataType variable = (dataType) value;
```

Type	Name	Conversion Direction	Example	Data Loss
Implicit	Widening	Small → Large	`int → double`	✗ No
Explicit	Narrowing	Large → Small	`double → int`	✓ Yes

10. IF-ELSE STATEMENTS

- The **if-else statement** allows your program to **make decisions** and execute code **based on conditions**.
- It helps your program **choose different paths** depending on whether a condition is *true* or *false*.
- Basic Syntax:

```
if (condition) {
   // code executes if condition is true
} else {
   // code executes if condition is false }
```

- Example:

```
int age = 18;
if (age >= 18) {
   System.out.println("You are eligible to vote.");
} else
{
   System.out.println("You are not eligible to vote.");
}
```

11. SWITCH CASE STATEMENTS

- The **switch statement** is used when you want to **compare one variable** against **multiple possible values**.
- It's a **cleaner alternative** to long *if-else-if ladders*.
- Syntax:

```
switch (expression) {
            case value1:
              // code block
              break;
            case value2:
            // code block
             break;
    default:
      // code block if no case matches
}
```

Example:

```java
package com.basic;

public class SwitchDemo
{
    public static void main(String[] args)
    {
        int no=1;
        switch(no)
        {
            case 1:
                System.out.println("YOUR NO IS ONE....");
                break;
            case 2:
                System.out.println("YOUR NO IS TWO....");
                break;
            case 3:
                System.out.println("YOUR NO IS THREE....");
                break;
            case 4:
                System.out.println("YOUR NO IS FOUR....");
                break;
            default:
                System.out.println("INVALID NO....");
                break;
        }
    }
}
```

12. LOOPS (FOR, WHILE, DO-WHILE)

- A **loop** is used to **repeat** a block of code **multiple times** until a condition becomes false.
- For Loop:

- Syntax:

```
for(initialization; condition; update)
{
   // code to repeat
}
```

- Example:

```
for(int i = 1; i <= 5; i++)
 {
    System.out.println("Hello " + i);
 }
```

- While Loop:

Syntax:

```
while(condition)
{
   // code to repeat
   // update (increment/decrement)
}
```

- Example:

```
int i = 1;
while(i <= 5) {
   System.out.println("Hello " + i);
   i++;
}
```

- Do-While Loop:

  - Syntax:

```
do
{
   // code to repeat
   // update
} while(condition);
```

  - Example:

```
int i = 1;
do {
   System.out.println("Hello " + i);
   i++;
} while(i <= 5);
```

13. BREAK AND CONTINUE KEYWORDS

- In Java, **jump statements** are used to **change the normal flow** of control in a program. Two main jump statements used in loops are:

    1. break
    2. Continue

- Both are used inside **loops** (for, while, do-while) and **switch** statements.

1. `break` Keyword:

- The `break` statement is used to immediately terminate (stop) the loop or switch statement.
- When Java encounters a break, it **jumps out of the loop/switch block completely** and moves to the next statement after the loop.

Example:

```
for(int i = 1; i <= 5; i++) {
   if(i == 3) {
      break;  // loop stops when i = 3
   }
   System.out.println("i = " + i);
}
```

2. continue Keyword

- The **continue** statement **skips the current iteration** of the loop and jumps to the **next iteration**.

Example:

```
for(int i = 1; i <= 5; i++) {
   if(i == 3) {
      continue; // skip this iteration
   }
   System.out.println("i = " + i);
}
```

14. DEFINING A CLASS AND OBJECT IN JAVA

- A **class** is a **blueprint** or **template** that defines how an **object** will behave and what it will contain.
- It groups **data (variables)** and **functions (methods)** together.
- Syntax of a Class:

```
class ClassName {
    // data members (variables)
    // methods (functions)
}
```

- Example:

```
class Student {
    // Data members (variables)
    int rollNo;
    String name;
    // Method (function)
    void showDetails() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}
```

- Object is an **instance** of a class — meaning it actually holds data and can call methods.
- Syntax to Create an Object:

```
ClassName objectName = new ClassName();
```

- Example**:**

```
class Student {
   int rollNo;
   String name;
   void showDetails()
{
      System.out.println("Roll No: " + rollNo);
      System.out.println("Name: " + name);
   }
```

```
   public static void main(String[] args) {
      // Creating Object
      Student s1 = new Student();
      // Assigning values
      s1.rollNo = 101;
      s1.name = "Shifa";
      // Calling method using object
      s1.showDetails();
   }
}
```

15. CONSTRUCTORS AND OVERLOADING

- A constructor is a **special method** in Java.
- It is used to **initialize the object** when it is created.
- It has the **same name as the class**.
- It **has no return type** (not even void).
- **Example:**

```
class Student
{
   Student() {          // Constructor
      System.out.println("Constructor Called");
   }
}
Student s1 = new Student(); // Constructor runs here
```

- Types of Constructors:

Type	Meaning	Example
Default Constructor	Java gives it automatically if we don't create any	`Student(){}` (empty)
No-Argument Constructor	Constructor without parameters	`Student(){...}`
Parameterized Constructor	Constructor with parameters to assign values	`Student(int id, String name){...}`

- When a class has **multiple constructors with the same name but different parameter lists,** it is called **Constructor Overloading**.

- Example:

```java
//PROGRAM FOR CONSTRUCTOR OVERLOADING DEMO
package com.oops;
class Sum
{
    int x, y;
    public Sum() // DEFAULT CONSTRUCTOR
    {
        x = 10;
        y = 20;
        System.out.println("FIRST SUM IS " + (x + y));
    }
    public Sum(int a) // SINGLE PARAMETERIZED CONSTRUCTOR
    {
        x = y = a;
        System.out.println("SECOND SUM IS " + (x + y));
    }
    public Sum(int a, int b) // MULTIPLE PARAMETERIZED CONSTRUCTOR
    {
        x = a;
        y = b;
        System.out.println("THIRD SUM IS " + (x + y));
    }
}
```

```java
public class ConstructorDemo
{
    public static void main(String[] args)
    {
        Sum sc = new Sum();
        new Sum(10);
        new Sum(10, 20);
//      System.gc();    //FIRST IT CALL FINALIZE AND THEN RELEASE VALUE

    }

}
```

16. OBJECT CREATION, ACCESSING MEMBERS OF THE CLASS

- An **object** is a **real-world entity** and an **instance of a class**.
- It represents the data (variables) and behavior (methods) of a class.
- In Java, an object is created using the **new keyword**.
- Syntax:

```
ClassName objectName = new ClassName();
```

- Class members are:

  1. **Variables (data members)**
  2. **Methods (member functions)**

- They are accessed using the **dot (.) operator**.

Accessing Variables:

```
objectName.variableName;
```

Example:

```
s1.id = 101;
s1.name = "shifa";
```

Accessing Methods:

```
objectName.methodName();
```

Example:

```
s1.display();
```

---

17. THIS KEYWORD

- this is a **reference variable**
- Refers to the **current object** of the class
- Used to **differentiate instance variables from local variables**
- Used inside **instance methods and constructors**
- Cannot be used inside **static methods**
- Can be used to **call current class constructor** (this())

- Can be used to **call current class method**
- Can be used to **pass current object as parameter**
- Can be used to **return current object**
- Improves **code clarity and readability**

```java
package com.keyword;

class TDemo
{
    int rno;
    String name;
    public void setData(int rno,String name)
    {
        this.rno=rno;
        this.name=name;
    }
    public void display()
    {
        System.out.println("ROLL NO IS "+rno);
        System.out.println("NAME  IS "+name);
    }
}

public class ThisDemo
{
    public static void main(String[] args)
    {
        TDemo t1=new TDemo();
        t1.setData(10,"SHIFA");
        t1.display();

    }
}
```

18. DEFINING METHODS

- A **method** is a block of code used to perform a **specific task**
- Methods are defined **inside a class**
- A method helps in **code reusability**
- It avoids **repetition of code**
- Methods make the program **modular**
- A method is executed only when it is **called**
- Methods improve **readability and maintainability**

- Method Definition Syntax:

```
returnType methodName(parameterList)
{
  // method body
}
```

Example:

```
class MethodDemo
{
   // Method definition
   int add(int a, int b) {
      return a + b;
   }
   public static void main(String[] args)
{
      MethodDemo obj = new MethodDemo();   // object creation
      int result = obj.add(10, 20);       // method call
      System.out.println("Sum = " + result);
   }
}
```

19. METHOD PARAMETERS AND RETURN TYPES

- Method parameters are used to pass values to a method
- Parameters act as input for the method
- A method can have zero, one, or multiple parameters
- Return type specifies the value returned by a method
- return keyword is used to return a value
- A method can return only one value
- void return type means no value is returned

20. METHOD OVERLOADING

- **Method overloading** means defining **more than one method with the same name** in the same class, but with **different parameters**.
- It allows a method to perform similar operations with different inputs and improves code readability.
- Method overloading is an example of **compile-time polymorphism**.

```java
//PRORAM TO PERFORM METHOD OVERLOADING
package com.oops;
class Addition
{
    int x,y;
    public void sum()
    {
        x=10;
        y=20;
        System.out.println("MY FIRST ADDITION IS "+(x+y));
    }
    public void sum(int a)
    {
        x=y=a;
        System.out.println("MY SECOND ADDITION IS "+(x+y));
    }
    public void sum(int a,int b)
    {
        x=a;
        y=b;
        System.out.println("MY THIRD ADDITION IS "+(x+y));
    }
    public void sum(float a)
    {
        x=y=(int)a;
        System.out.println("MY FOURTH ADDITION IS "+(x+y));
    }
}
```

```java
public class MethodOverloading
{
    public static void main(String[] args)
    {
        Addition a1=new Addition();
        a1.sum();
        a1.sum(10);
        a1.sum(10,20);
        a1.sum(10.30f);
    }
}
```

21. STATIC METHODS AND VARIABLES

- **Static methods and variables** are members of a class that belong to the **class itself,** rather than to any specific object.
- This means they can be accessed **without creating an object** of the class.

Static Variables

- Declared using the static keyword
- Shared by **all objects** of the class
- Changes made by one object are **reflected in all objects**
- Used to store **common properties** of all objects

Example:

```
class Student {
   static String school = "ABC School"; // static variable
   String name;
   Student(String name)
 {
      this.name = name;
   }
   void display()
{
      System.out.println(name + " studies at " + school);
   }
```

Static Methods

- Declared using the static keyword
- Can be called **without creating an object**
- Can access **only static variables and other static methods**
- Cannot use this keyword

```
class MathUtil {
   static int square(int n)
{
      return n * n;
 }
   public static void main(String[] args)
{
      System.out.println(MathUtil.square(5)); // calling static method without
object
  } }
```

22. Basics of OOP: Encapsulation, Inheritance, Polymorphism, Abstraction

✓ **Encapsulation**

- Encapsulation is the process of **wrapping data (variables) and methods** that operate on that data into a **single unit (class)**.
- It **hides the internal details** of an object from the outside world.
- Achieved using **private variables** and **public getter/setter methods**.

✓ **Inheritance**

- Inheritance allows a class to **acquire properties and behavior** of another class.
- Helps in **code reusability**.
- Achieved using the **extends** keyword.

✓ **Polymorphism**

- Polymorphism means **one name, many forms**.
- Allows objects to **behave differently** based on context.
- Types:

  - **Compile-time (Method Overloading)**
  - **Run-time (Method Overriding)**

✓ **Abstraction**

- Abstraction hides **implementation details** and shows only the **essential features**.
- Achieved using **abstract classes** or **interfaces**.

23. INHERITANCE: SINGLE, MULTILEVEL, HIERARCHICAL

- **Inheritance** is a feature of OOP where a class **acquires properties and methods** of another class.

  - The class which **inherits** is called **subclass/child class**
  - The class whose properties are inherited is called **superclass/parent class**
  - Helps in **code reusability**

1.Single Inheritance

A subclass inherits from **one superclass** only.

Example:

```
class Animal
{
   void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
   void bark() { System.out.println("Barking"); }
}
```

2.Multilevel Inheritance

- A class inherits from a subclass**,** forming a chain of inheritance**.**
- Example**:**

```
class Animal
{
   void eat() { System.out.println("Eating"); }
}
class Mammal extends Animal
{
   void walk() { System.out.println("Walking"); }
}
class Dog extends Mammal
{
   void bark() { System.out.println("Barking"); }
}
```

3. Hierarchical Inheritance

- **Multiple subclasses** inherit from one superclass.

```
class Animal {
   void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
   void bark() { System.out.println("Barking"); }
}
class Cat extends Animal {
   void meow() { System.out.println("Meowing"); }
}
```

24. METHOD OVERRIDING AND DYNAMIC METHOD DISPATCH

✓ Method Overriding

- **Method overriding** occurs when a **subclass provides its own implementation** of a method that is already defined in its **superclass**.
- The **method name, return type, and parameters** must be **exactly the same**.
- It allows a subclass to **modify or extend the behavior** of a superclass method.
- Overriding is an example of **runtime polymorphism**.

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal makes sound");
   }
}
class Dog extends Animal {
   void sound() {
      System.out.println("Dog barks");
   }
}
```

✓ Dynamic Method Dispatch

- **Dynamic Method Dispatch** is the process by which a **call to an overridden method** is resolved **at runtime** rather than compile time.
- It allows **polymorphic behavior,** where a **superclass reference** can point to a **subclass object** and call the overridden method.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();  // superclass reference, subclass object
        a.sound();          // calls Dog's overridden method at runtime
    }
}
```

25. CONSTRUCTOR TYPES (DEFAULT, PARAMETERIZED)

- A **constructor** is a special method used to **initialize objects** of a class. It has the **same name as the class** and **no return type**.

1. Default Constructor

- A constructor with **no parameters**
- Initializes objects with **default values**
- Java provides a **default constructor** if no constructor is defined

2. Parameterized Constructor

- A constructor that **accepts parameters**
- Used to initialize objects with **specific values**

```java
//PROGRAM FOR CONSTRUCTOR OVERLOADING DEMO
package com.oops;
class Sum
{
    int x, y;
    public Sum() // DEFAULT CONSTRUCTOR
    {
        x = 10;
        y = 20;
        System.out.println("FIRST SUM IS " + (x + y));
    }
    public Sum(int a) // SINGLE PARAMETERIZED CONSTRUCTOR
    {
        x = y = a;
        System.out.println("SECOND SUM IS " + (x + y));
    }
    public Sum(int a, int b) // MULTIPLE PARAMETERIZED CONSTRUCTOR
    {
        x = a;
        y = b;
        System.out.println("THIRD SUM IS " + (x + y));
    }
}
```

26. COPY CONSTRUCTOR (EMULATED IN JAVA)

- A **copy constructor** is a constructor that creates a **new object by copying the values** of an existing object.
- Java **does not provide a built-in copy constructor,** but it can be **emulated by creating a constructor that takes an object of the same class** as a parameter.
- Useful when you want to **duplicate objects with the same data**.

Example:

```
class Student {
   int id;
   String name;
   // Parameterized constructor
   Student(int i, String n) {
      id = i;
      name = n;
   }
   // Copy constructor (emulated)
   Student(Student s) {
      id = s.id;
      name = s.name;
   }
   void display() {
      System.out.println(id + " " + name);
   }
}
```

```
  public static void main(String[] args) {
        Student s1 = new Student(101, "Shifa");
        Student s2 = new Student(s1); // creating copy using copy
  constructor
        s1.display();
        s2.display();
     }
  }
```

27. CONSTRUCTOR OVERLOADING

```java
//PROGRAM FOR CONSTRUCTOR OVERLOADING DEMO
package com.oops;
class Sum
{
    int x, y;
    public Sum() // DEFAULT CONSTRUCTOR
    {
        x = 10;
        y = 20;
        System.out.println("FIRST SUM IS " + (x + y));
    }
    public Sum(int a) // SINGLE PARAMETERIZED CONSTRUCTOR
    {
        x = y = a;
        System.out.println("SECOND SUM IS " + (x + y));
    }
    public Sum(int a, int b) // MULTIPLE PARAMETERIZED CONSTRUCTOR
    {
        x = a;
        y = b;
        System.out.println("THIRD SUM IS " + (x + y));
    }
}
```

```java
public class ConstructorDemo
{
    public static void main(String[] args)
    {
        Sum sc = new Sum();
        new Sum(10);
        new Sum(10, 20);
//      System.gc();    //FIRST IT CALL FINALIZE AND THEN RELEASE VALUE

    }

}
```

28. OBJECT LIFE CYCLE AND GARBAGE COLLECTION

✓ Object Life Cycle:

The **object life cycle** describes the **stages an object goes through** from creation to destruction:

1. **Creation** – Object is created using the new keyword and memory is allocated on the heap.
2. **Use** – Object's methods and variables are accessed and used.
3. **Become Unreachable** – When there are no references pointing to the object**,** it becomes eligible for garbage collection.
4. **Destruction** – Object is destroyed and memory is freed by the garbage collector.

✓ Garbage Collection:

- **Garbage Collection (GC)** is the process by which **Java automatically removes objects** that are no longer needed.
- Helps to **free memory and avoid memory leaks**.
- The **finalize() method** can be used to perform cleanup before an object is destroyed (deprecated in modern Java).
- **Garbage collection is automatic;** calling System.gc() **only suggests** it**,** but does not guarantee immediate execution.

29. One-Dimensional and Multidimensional Arrays

- An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**. Arrays are used to store multiple values in a **single variable**.

✓ One-Dimensional Array

- Stores elements in a **single row**
- Accessed using a **single index**
- Useful for storing **linear data**

Example:

```
int[] numbers = {10, 20, 30, 40};

System.out.println(numbers[2]);  // Output: 30
```

✓ Multidimensional Array

- Stores elements in **rows and columns** (2D or more)
- Accessed using **multiple indices**
- Useful for storing **tabular or matrix data**

Example:

```
int[][] matrix = { {1, 2}, {3, 4} };
System.out.println(matrix[1][0]); // Output: 3
```

30. String Handling in Java: String Class, String Buffer, String Builder

- In Java, strings are used to store text data. Java provides three main ways to handle strings: **String**, **StringBuffer**, and **StringBuilder**.

String Class

- Represents a **sequence of characters**
- Strings are **immutable**, meaning their value **cannot be changed** once created
- Any operation that seems to change a string actually creates a **new string**
- Methods: `length()`, `charAt()`, `concat()`, `substring()`, `equals()`

StringBuffer Class

- Used to create **mutable strings** (can be changed)
- Thread-safe (synchronized)
- Slower than StringBuilder due to synchronization
- Methods: append(), insert(), replace(), delete(), reverse()

StringBuilder Class

- Similar to StringBuffer, but **not synchronized** (faster)
- Used for **mutable strings**
- Suitable when **thread-safety is not required**
- Methods are same as StringBuffer

31. Array of Objects

- An **array of objects** is an array that stores **multiple objects of the same class**.
- Each element of the array is a **reference to an object**.
- Useful when we need to **handle multiple objects together** in a single data structure.

Example**:**

```
class Student {
   int id;
   String name;
   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
   void display() {
      System.out.println(id + " " + name);
   }
}
```

```
 public class Test {
    public static void main(String[] args) {
       Student[] arr = new Student[3]; // array of objects
       arr[0] = new Student(101, "Shifa");
       arr[1] = new Student(102, "Rahul");
       arr[2] = new Student(103, "Amit");
       for(int i=0; i<arr.length; i++) {
          arr[i].display();
       }
    }
 }
```

32. String Methods (length, charAt, substring, etc.)

i) length()

- Returns the **number of characters** in the string

```
String s = "Hello";
System.out.println(s.length()); // Output: 5
```

ii) charAt(int index)

- Returns the **character** at the specified index

```
String s = "Hello";
System.out.println(s.charAt(1)); // Output: e
```

iii) substring(int beginIndex, int endIndex)

- Returns a **part of the string** from beginIndex to endIndex-1

```
String s = "Hello";
System.out.println(s.substring(1,4)); // Output: ell
```

iv) concat(String str)

- Concatenates (joins) two strings

```
String s = "Hello";
System.out.println(s.concat(" World")); // Output: Hello World
```

33. INHERITANCE TYPES AND BENEFITS

- **Inheritance** is a feature of OOP that allows a class to **acquire properties and methods** of another class.

  - The class whose properties are inherited is called the **superclass/parent class**.
  - The class that inherits is called the **subclass/child class**.
  - Inheritance promotes **code reusability** and **modular programming**.

Types of Inheritance

1. **Single Inheritance**
   - A subclass inherits from **one superclass**.
   - Example: class Dog extends Animal { }
2. **Multilevel Inheritance**
   - A class inherits from a **subclass,** forming a chain.
   - Example: class Dog extends Mammal extends Animal { }
3. **Hierarchical Inheritance**
   - **Multiple subclasses** inherit from **one superclass**.
   - Example: class Dog extends Animal { }, class Cat extends Animal { }
4. **Multiple Inheritance (via Interface)**
   - Java **does not support multiple inheritance with classes,** but can be achieved using **interfaces**.

Benefits of Inheritance

- **Code Reusability:** Avoids rewriting common code
- **Extensibility:** Easy to add new features
- **Data Hiding and Security:** Superclass can keep variables private
- **Polymorphism Support:** Enables method overriding for runtime polymorphism
- **Maintainability:** Changes in superclass automatically affect subclasses

34. METHOD OVERRIDING

- **Method overriding** occurs when a **subclass provides its own implementation** of a method that is already defined in its **superclass**.
- The **method name, return type, and parameters** must be **exactly the same** in both superclass and subclass.
- It allows a subclass to **modify or extend the behavior** of the superclass method.
- Method overriding is an example of **runtime polymorphism**.

```java
//PROGRAM TO PERFORM METHOD OVERRIDING
package com.oops;
class First
{
    public void display()
    {
        System.out.println("THIS IS FIRST CLASS METHOD...");
    }
}
class Second extends First
{
    public void display()
    {
        System.out.println("THIS IS SECOND CLASS METHOD...");
    }
}
public class MethodOverriding
{
    public static void main(String[] args)
    {
//      Second s1=new Second();
//      s1.display();
        First f1=new First();
        f1.display();
        f1=new Second();
        f1.display();
    }
}
```

35. DYNAMIC BINDING (RUN-TIME POLYMORPHISM)

- **Dynamic binding** occurs when a **method call is resolved at runtime** rather than at compile time.
- It is achieved using **method overriding**, where a **subclass provides its own implementation** of a method defined in the superclass.
- It allows a **superclass reference** to refer to a **subclass object**, enabling **polymorphic behavior**.

```java
//PROGRAM TO PERFORM METHOD OVERRIDING
package com.oops;
class First
{
    public void display()
    {
        System.out.println("THIS IS FIRST CLASS METHOD...");
    }
}
class Second extends First
{
    public void display()
    {
        System.out.println("THIS IS SECOND CLASS METHOD...");
    }
}
public class MethodOverriding
{
    public static void main(String[] args)
    {
//      Second s1=new Second();
//      s1.display();
        First f1=new First();
        f1.display();
        f1=new Second();
        f1.display();
    }
}
```

36. SUPER KEYWORD AND METHOD HIDING

Super Keyword

- The super keyword refers to the **immediate parent class** of a subclass.
- It is used to:

    1. Access **parent class variables** when they are hidden by subclass variables
    2. Call **parent class methods**
    3. Call **parent class constructor**

Method Hiding in Java

- **Method hiding** occurs when a **static method** in a subclass **has the same name** as a static method in the superclass.
- Unlike method overriding, **static methods are bound at compile-time,** not runtime.
- The **method called depends on the reference type,** not the object type.

37. Abstract Classes and Methods

✓ Abstract Class

- An **abstract class** is a class that **cannot be instantiated** directly.
- Declared using the abstract keyword.
- Can contain **both abstract methods (without body) and concrete methods (with body)**.
- Provides a **common structure** for related subclasses.

```
abstract class Shape
{
   void welcome() {          // concrete method
      System.out.println("Welcome to Shape Drawing");
   }
}
```

✓ Abstract Method

- An **abstract method** is a method **declared without implementation**.
- Declared using the abstract keyword.
- Must be **overridden in a subclass**.
- Cannot have a **method body** in the abstract class.

```
abstract class Shape {
   abstract void draw(); // abstract method
}
class Circle extends Shape {
   void draw() { // overriding abstract method
      System.out.println("Drawing Circle");  }
}
```

38. Interfaces: Multiple Inheritance in Java

✓ Interfaces in Java

- An **interface** is a blueprint of a class.
- It contains **abstract methods** (by default) and **constants**.
- Interfaces are declared using the interface keyword.
- A class uses an interface using the **implements** keyword.

✓ Multiple Inheritance in Java using Interface

- Java **does not support multiple inheritance with classes** to avoid ambiguity.
- Java **supports multiple inheritance using interfaces**.
- A class can **implement multiple interfaces** at the same time.
- This allows a class to inherit behavior from **more than one source**.

Syntax:

```
class ClassName implements Interface1, Interface2
{
   // method implementations
}
```

39. IMPLEMENTING MULTIPLE INTERFACES

- Java allows a class to **implement more than one interface**.
- This helps achieve **multiple inheritance,** which is not possible with classes.
- A class must **provide implementations for all methods** of the interfaces it implements.
- Multiple interfaces are separated using **comma (,)**.

Example:

```java
package com.basic;

interface Printable
{
    void print();
}
interface Showable
{
    void show();
}
public class Multiple_Interface implements Printable, Showable
{
      public void print()
      {
            System.out.println("Printing");
      }

      public void show()
      {
            System.out.println("Showing");
      }
    public static void main(String[] args)
    {
        Multiple_Interface d = new Multiple_Interface();
            d.print();
            d.show();
    }
```

40. JAVA PACKAGES: BUILT-IN AND USER-DEFINED PACKAGES

➢ A **package** in Java is a group of **related classes and interfaces**.

➢ Packages help in **organizing code, avoiding name conflicts,** and **improving security**.

Built-in Packages

- Built-in packages are **predefined packages provided by Java**.
- They contain **ready-made classes and interfaces**.
- Used by importing them into a program using the import keyword.

Examples of Built-in Packages

- java.lang → basic classes (String, Math, System)
- java.util → utility classes (Scanner, ArrayList)
- java.io → input/output classes
- java.sql → database connectivity

User-Defined Packages

- User-defined packages are **created by programmers**.
- Used to **organize large projects**.
- Created using the package keyword.

Syntax:

```
package packagename;
```

Example:

```
package com.example;
public class Demo
{
   public void show()
{
      System.out.println("User-defined package");
   }
}
```

41. Access Modifiers: private, default, protected, public

- **Access modifiers** define the **visibility and accessibility** of classes, methods, and variables in Java.
- Java provides four types of access modifiers: **private, default, protected, and public**.

Private

- Accessible **only within the same class**
- Provides the highest level of data hiding
- Cannot be accessed outside the class

Default (No Modifier)

- Accessible **within the same package only**
- Also called **package-private**
- Used when no access modifier is specified

Protected

- Accessible within the **same package**
- Also accessible in **subclasses outside the package**
- Used mainly in **inheritance**

Public

- Accessible **from anywhere**
- Provides the **least restriction**
- Used for classes and methods meant for general use

42. Importing Packages and Classpath

Importing Packages

- **Importing packages** allows a Java program to **use classes and interfaces** defined in other packages.
- The import keyword is used to access **built-in or user-defined packages**.
- It helps avoid writing the **fully qualified class name** every time.

```
import packageName.ClassName;

import packageName.*;
```

Classpath in Java

- Classpath is the **path where the Java compiler and JVM look for classes and packages**.
- It tells Java where to find user-defined and built-in classes**.**
- If a class is not found in the classpath**,** Java throws a ClassNotFoundException**.**

43. Types of Exceptions: Checked and Unchecked

- An **exception** is an event that occurs during program execution and **disrupts the normal flow** of the program.

✓ Checked Exceptions

- Checked exceptions are **checked at compile time**
- They must be **handled using try–catch** or **declared using throws**

- Occur due to **external factors**

Examples:

- IOException
- SQLException
- FileNotFoundException

✓ Unchecked Exceptions

- Unchecked exceptions are **checked at runtime**
- They are **not compulsory** to handle
- Usually occur due to **programming errors**

Examples:

- NullPointerException
- ArithmeticException
- ArrayIndexOutOfBoundsException

44. try, catch, finally, throw, throws

try

- The try block contains **code that may cause an exception**.
- It must be followed by **at least one catch or a finally block.**

catch

- The catch block **handles the exception** thrown in the try block.
- It prevents the program from terminating abnormally.

finally

- The finally block **always executes,** whether an exception occurs or not.
- Used for **cleanup code** like closing files or database connections.

throw

- The throw keyword is used to **explicitly throw an exception**.
- Used inside a method or block.

throws

- The throws keyword is used to **declare exceptions** in a method signature.

- It passes the responsibility of handling the exception to the **calling method**.

45. Custom Exception Classes

- A **custom exception** is a **user-defined exception** created by extending the Exception or RuntimeException class.
- It allows programmers to **define their own error conditions**.
- Used when built-in exceptions are **not sufficient**.

Creating a Custom Exception (Checked)

```
class InvalidAgeException extends Exception
{
  InvalidAgeException(String msg) {
     super(msg);
  }
}
```

46. Introduction to Threads

- A **thread** is a **smallest unit of execution** within a program.
- Java supports **multithreading,** which allows multiple threads to run **simultaneously**.
- Threads help in **better CPU utilization** and **faster execution** of programs.
- Each thread runs **independently** but shares the same memory space.
- Every Java program starts with a **main thread**.
- Other threads are created from the main thread.

Ways to Create a Thread

1. **Extending Thread class**
2. **Implementing Runnable interface**

47. Creating Threads by Extending Thread Class or Implementing Runnable Interface

Extending the Thread Class

- Create a class that **extends Thread**

- Override the run() method
- Call start() to begin execution

```java
package com.ThreadDemo;

class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread running using Thread class");
    }
}

public class ThreadExample
{
    public static void main(String[] args)
    {
          MyThread t = new MyThread();
          t.start();
    }
}
```

Implementing the Runnable Interface

- Create a class that **implements Runnable**
- Override the run() method
- Pass the object to a Thread constructor
- Call start()

```java
package com.ThreadDemo;

class MyRunnable implements Runnable
{
    public void run()
    {
        System.out.println("Thread running using Runnable interface");
    }
}
public class RunnableExample
{
    public static void main(String[] args)
    {
        MyRunnable r=new MyRunnable();
        Thread t = new Thread(r);
        t.start();
    }
}
```

48. Thread Life Cycle

- The **Thread Life Cycle** describes the different **states** a thread goes through from creation to termination.

1. New State

- A thread is created but **not started yet**.

2. Runnable State

- Thread is **ready to run**.
- After calling start(), thread enters this state.

3. Running State

- Thread is **executing its task**.
- JVM scheduler selects the thread.
- Executes the run() method.

4. Blocked / Waiting State

Thread is **temporarily inactive**.

5. Terminated (Dead) State

- Thread **finishes execution** or is stopped.
- Cannot be restarted.

49. Synchronization and Inter-Thread Communication

Synchronization

Synchronization is a mechanism in Java used to **control access to shared resources** so that **only one thread can access a resource at a time**.

Need for Synchronization

- Multiple threads may access the same object simultaneously
- Prevents **race condition**
- Maintains **data consistency**

Working of Synchronization

- Java uses **locks (monitors)** for synchronization
- When a thread enters a synchronized area, it **acquires a lock**

- Other threads must **wait** until the lock is released

Inter-thread Communication

Inter-thread communication is a mechanism that allows **threads to communicate and cooperate** with each other while executing.

Purpose

- Allows threads to **wait and notify** each other
- Avoids **busy waiting**
- Used when one thread depends on another

Methods Used

All methods belong to the **Object class**:

- wait()
- notify()
- notifyAll()

How It Works

- A thread calls wait() and **releases the lock**
- Another thread calls notify() or notifyAll()
- Waiting thread resumes execution after getting the lock

50. Introduction to File I/O in Java (java.io package)

- **File I/O (Input/Output)** in Java is used to **read data from a file** and **write data to a file**.
- The java.io package provides classes that allow Java programs to **perform file handling operations**.

Why File I/O is Needed

- To **store data permanently**
- To **read and write files**
- To handle **large amounts of data**
- To exchange data between programs

java.io Package

The java.io package contains classes for:

- File handling

- Byte stream I/O
- Character stream I/O
- Object serialization

Streams in Java

A **stream** is a flow of data from source to destination.

Types of Streams

1. Byte Streams

- Used to handle **binary data** (images, audio, video)
- Classes:
  - InputStream
  - OutputStream

2. Character Streams

- Used to handle **text data**
- Classes:
  - Reader
  - Writer

File Class

The File class represents a **file or directory** in the system.

Uses

- Create a file
- Delete a file
- Check file properties

51. FileReader and FileWriter Classes

- FileReader and FileWriter are **character stream classes** present in the **java.io package**.
- They are used to **read and write text data** from and to files.

Key Points

- Reads data **character by character**
- Suitable for **text files**
- Works with **Unicode characters**

- Subclass of Reader

Purpose

- To read text data such as letters, words, and lines from a file

Features

- Converts bytes into characters automatically
- Supports different character encodings

FileWriter Class

- **FileWriter** is used to **write characters** into a text file.

Key Points

- Writes data **character by character**
- Creates a file if it does not exist
- Overwrites file content by default
- Subclass of Writer

Purpose

- To write text data into a file

52. BufferedReader and BufferedWriter

- BufferedReader and BufferedWriter are **character stream classes** in the **java.io package**.
- They are used to **read and write text data efficiently** by using a **buffer (temporary memory)**.

BufferedReader

- **BufferedReader** is used to **read text from an input stream efficiently**.

Key Features

- Reads data **line by line**
- Uses a buffer to reduce I/O operations
- Faster than FileReader
- Subclass of Reader

Purpose

- To read large text files efficiently

Important Method

- readLine() → reads one line at a time

BufferedWriter

- **BufferedWriter** is used to **write text to an output stream efficiently**.

Key Features

- Writes data using a buffer
- Improves performance
- Subclass of Writer

Purpose

- To write large text data efficiently

Important Methods

- write()
- newLine()
- flush()

53. SERIALIZATION AND DESERIALIZATION

Serialization

- **Serialization** is the process of **converting an object into a byte stream** so that it can be **stored in a file or transmitted over a network**.

  - Object state is saved
  - Uses java.io package
  - Class must implement **Serializable interface**

Deserialization

- **Deserialization** is the process of **converting a byte stream back into an object**.

  - Restores object state
  - Reads object data from file or network

54. INTRODUCTION TO COLLECTIONS FRAMEWORK

- The **Java Collections Framework (JCF)** is a **set of classes and interfaces** used to **store, manipulate, and manage groups of objects** efficiently.
- It is available in the **java.util package**.

Why Collections Framework is Needed

- Arrays have **fixed size**
- No built-in methods for searching, sorting, or insertion
- Collections provide **dynamic size**
- Ready-made methods for data manipulation

Components of Collections Framework

1. Interfaces

Define the blueprint for collection classes.

- Collection
- List
- Set
- Queue
- Map

2. Classes

Provide implementation of interfaces.

- ArrayList
- LinkedList
- HashSet
- TreeSet
- HashMap
- TreeMap

55. LIST, SET, MAP, AND QUEUE INTERFACES

- The **List, Set, Map, and Queue** are the **core interfaces** of the **Java Collections Framework**, present in the java.util package.
- They define different ways to **store and manage collections of objects**.

1. List Interface

- The **List interface** represents an **ordered collection** of elements.

Characteristics

- Allows **duplicate elements**
- Maintains **insertion order**
- Elements can be accessed by index

Common Implementations

- ArrayList
- LinkedList
- Vector

2. Set Interface

The **Set interface** represents a **collection of unique elements**.

Characteristics

- Does **not allow duplicate elements**
- Order is not guaranteed (except LinkedHashSet)
- No index-based access

Common Implementations

- HashSet
- LinkedHashSet
- TreeSet

3. Map Interface

➢ The **Map interface** stores data in the form of **key–value pairs**.

Characteristics

- Keys are **unique**
- Values can be duplicated
- Does **not extend Collection interface**

Common Implementations

- HashMap
- LinkedHashMap
- TreeMap
- Hashtable

4. Queue Interface

The **Queue interface** represents a collection designed for **holding elements before processing**.

Characteristics

- Follows **FIFO (First In, First Out)**
- Elements are inserted at the end and removed from the front
- Some queues support priority-based ordering

Common Implementations

- PriorityQueue
- ArrayDeque

56. ARRAYLIST, LINKEDLIST, HASHSET, TREESET, HASHMAP, TREEMAP

i) ArrayList

➢ ArrayList is a **resizable array implementation** of the **List interface**.

Characteristics

- Allows **duplicate elements**
- Maintains **insertion order**
- Provides **fast random access**
- Not synchronized

Use Case

- When frequent **reading/accessing** of elements is required

ii) LinkedList

➢ LinkedList is a **doubly linked list implementation** of the **List and Deque interfaces**.

Characteristics

- Allows **duplicate elements**
- Maintains **insertion order**
- Slower access than ArrayList
- Faster insertion and deletion

Use Case

- When frequent **insertion and deletion** operations are required

iii) HashSet

➢ HashSet is an implementation of the **Set interface**.

Characteristics

- Does **not allow duplicates**
- Does not maintain order
- Uses **hashing**
- Allows one null element

Use Case

- When **uniqueness** is required and order does not matter

iv) TreeSet

➢ TreeSet is an implementation of the **NavigableSet interface**.

Characteristics

- Does **not allow duplicates**
- Maintains elements in **sorted order**
- Does not allow null elements
- Slower than HashSet

Use Case

- When **sorted unique elements** are required

v) HashMap

➢ HashMap is an implementation of the **Map interface**.

Characteristics

- Stores data in **key–value pairs**
- Allows **one null key** and multiple null values
- No insertion order
- Fast performance

Use Case

- When fast lookup using keys is needed

vi) TreeMap

- TreeMap is an implementation of the **NavigableMap interface**.

Characteristics

- Stores data in **sorted order of keys**
- Does not allow null keys
- Slower than HashMap
- Values can be null

Use Case

- When **sorted key–value pairs** are required

57. Iterators and ListIterators

1. Iterator Interface

- **Iterator** is an interface in the **java.util package** used to **traverse elements** of a collection (like ArrayList, HashSet) **one by one**.

Key Points

- Works with **all collection classes** implementing the **Collection interface**
- **Can traverse only forward**
- Provides **read and remove** operations
- Does **not support adding elements**

Important Methods

Method	Description
hasNext()	Returns true if more elements are present
next()	Returns the next element
remove()	Removes the last element returned by next()

2. ListIterator Interface

- **ListIterator** is an interface used to **traverse elements of a List (like ArrayList, LinkedList) in both directions**.

Key Points

- Works only with **List interface implementations**
- Can traverse **forward and backward**
- Can **add, remove, or modify elements** during traversal

Important Methods

Method	Description
`hasNext()`	Checks if next element exists
`next()`	Returns next element
`hasPrevious()`	Checks if previous element exists
`previous()`	Returns previous element
`add(Object o)`	Adds an element
`remove()`	Removes current element
`set(Object o)`	Replaces current element

58. Streams in Java (InputStream, OutputStream)

- In Java, a **stream** is a **sequence of data** that can be read from or written to a source such as a **file, keyboard, or network**.
- Streams provide a **flow of data** from a **source (input)** to a **destination (output)**.

Types of Streams

1. **Byte Streams**
    - Handle **binary data** (images, audio, video)
    - Read/write **one byte at a time**
    - Base classes:
        - InputStream (for reading)
        - OutputStream (for writing)
2. **Character Streams**
    - Handle **text data**
    - Read/write **characters or strings**
    - Base classes:

- Reader (for reading)
- Writer (for writing)

1. InputStream

InputStream is an **abstract class** for reading **binary data** from a source.

Common Methods

Method	Description
read()	Reads **one byte** of data
read(byte[] b)	Reads bytes into an array
close()	Closes the stream

2. OutputStream

OutputStream is an **abstract class** for writing **binary data** to a destination.

Common Methods

Method	Description
write(int b)	Writes **one byte** of data
write(byte[] b)	Writes bytes from an array
flush()	Forces all data to be written out
close()	Closes the stream

59. Reading and Writing Data Using Streams

- In Java, **streams** are used to **read data from a source** (like a file, keyboard, or network) and **write data to a destination**.

1. Reading Data Using Streams

- ✓ **Using InputStream (Byte Stream)**
  - Reads **binary data** from a source.

- Example classes: FileInputStream, BufferedInputStream
- **Common Methods**:
  - read() → reads **one byte**
  - read(byte[] b) → reads bytes into an array
  - close() → closes the stream

Steps to Read

1. Create InputStream object
2. Read data using read() methods
3. Close the stream

✓ **Using Reader (Character Stream)**

- Reads **text data** (characters)
- Example classes: FileReader, BufferedReader
- **Important Method**:
  - read() → reads one character
  - readLine() (BufferedReader) → reads a line of text

2. Writing Data Using Streams

✓ **Using OutputStream (Byte Stream)**

- Writes **binary data** to a destination
- Example classes: FileOutputStream, BufferedOutputStream
- **Common Methods**:
  - write(int b) → writes one byte
  - write(byte[] b) → writes array of bytes
  - flush() → forces all data to be written
  - close() → closes the stream

✓ **Using Writer (Character Stream)**

- Writes **text data**
- Example classes: FileWriter, BufferedWriter
- **Important Methods**:
  - write(char[] c) → writes characters
  - newLine() → inserts a new line
  - flush() → flushes buffer
  - close() → closes the stream

60. HANDLING FILE I/O OPERATIONS

- **File I/O (Input/Output)** in Java allows a program to **read data from files** and **write data to files,** enabling **persistent storage**.
- All file handling classes are in the **java.io package**.

Key Classes for File I/O:

Class	Purpose
File	Represents a file or directory; used to create, delete, or check properties
FileReader	Reads characters from a file
FileWriter	Writes characters to a file
BufferedReader	Reads text efficiently using buffering
BufferedWriter	Writes text efficiently using buffering
FileInputStream	Reads bytes from a file (binary)
FileOutputStream	Writes bytes to a file (binary)